

Noms : KAMDOM NGUETCHESSI
Prénoms : Merveille Prisca
Matricule : 23V2060
Niveau : LICENCE 3 INFORMATIQUE
UE : INF 371 – BUSINESS INTELLIGENT

Rapport d'Étude de Cas : Conception d'un Entrepôt de Données pour « Les Débrouillards »

Introduction

Ce rapport présente l'analyse, la modélisation et la conception de l'entrepôt de données pour l'entreprise « Les Débrouillards », spécialisée dans la transformation et la vente de produits alimentaires. L'objectif est de mettre en place un système décisionnel basé sur le processus de **Vente** afin de répondre aux besoins analytiques exprimés par l'entreprise (analyse des ventes, suivi des performances, et évaluation des promotions). Ce document répond point par point aux exigences méthodologiques, de la définition des indicateurs (KPIs), à la proposition des schémas dimensionnels et à l'implémentation / Alimentation du dataWarehouse de cette entreprise .

A. Analyse et conception

1. Extension des besoins et objectifs analytiques

Suite à l'évolution de la structure des données opérationnelles, nous complétons la liste initiale par cinq questions stratégiques supplémentaires. Ces questions visent à transformer les données brutes en leviers de performance pour « Les Débrouillards »

N°	Question (Besoin métier)	Justification et Objectif Stratégique	Finalité Concrète
j	Quelle est la marge bénéficiaire nette par catégorie de produits et par région ?	Exploiter la nouvelle donnée CoutAchat pour ne plus seulement suivre le volume, mais la rentabilité réelle.	Identifier les produits qui génèrent le plus de profit après déduction des coûts et des remises.
k	Quel est le comportement d'achat par type de client (Particulier, Revendeur, Grossiste) ?	Segmenter l'analyse pour comprendre quel profil de client contribue le plus à la stabilité du chiffre d'affaires.	Adapter la politique de prix et les stocks en fonction de la dominance d'un segment sur une période.
l	Quelle est la performance comparée des canaux de vente (ModeCommande) en termes de volume ?	Analyser si l'investissement dans l'Application Mobile , Téléphone ou le Site Web est plus efficace que les ventes en Magasin.	Réallouer les ressources logistiques et marketing vers les canaux les plus porteurs et les moins porteurs pour attirer plus de clientèle .
m	Quel est l'impact réel des taux de promotion sur la marge brute ?	Vérifier si une remise élevée (TauxPromo) booste assez le volume pour compenser la perte de valeur unitaire.	Optimiser le seuil de rentabilité des campagnes promotionnelles (éviter les promotions à perte).
n	Quelle est la fidélité géographique des clients	Mesurer la récurrence des transactions par zone pour évaluer	Déterminer s'il faut ouvrir de nouveaux points de vente

	(répétition d'achat par Ville/Région) ?	la pénétration du marché local	physiques ou renforcer la livraison.
--	---	--------------------------------	--------------------------------------

2.Proposition des Indicateurs Clés de Performance (KPIs)

Le tableau suivant détaille les KPIs pour chaque question posée par l'entreprise, en précisant les mesures, les formules et les dimensions nécessaires pour l'analyse.

N°	Question (Libellé)	KPI	Objectif	Type	Mesures de Base	Formule de Calcul	Dimension(s)	Granularité
a	Quel est le montant total des ventes par région, par période et par produit ?	KPI-Q-a-01 : CA Net par Segment	Mesurer la contribution financière par produit et localisation.	Financier	-Quantité -Prix Unitaire, -TauxPromo	$SUM(Quantité * P.U. * (1 - TauxPromo/100))$	-Produit, -Temps, -Lieu	-Produit -> Catégorie - Mois -> Trimestre -> Année - Ville -> Region
b	Quels sont les produits les plus vendus par catégorie ?	KPI-Q-b-02 : Volume Total Vendu	Identifier les produits les plus demandés en volume pour la gestion des stocks.	Volume	Quantité	$SUM(Quantité)$	-Produit	-Produit -> Catégorie
c	Comment évoluent les ventes mensuelles dans les différentes villes ?	KPI-Q-c-03 : Taux de Croissance du CA Mensuel	Évaluer la dynamique commerciale et la progression de la demande par ville.	Taux/ Tendance	CA Net	$((CA \text{ du Mois } N / CA \text{ du Mois } N-1) - 1) * 100$	-Temps, -Lieu	-Mois -> Trimestre -> Année - Ville -> Region
d	Quels sont les produits les plus demandés dans les différentes villes / régions ?	KPI-Q-d-04 : Demande en Volume par Localisation	Identifier la demande spécifique (produit) selon la localisation géographique (villes/régions).	Volume	Quantité	$SUM(Quantité)$	-Produit - Lieu	-Produit -> Catégorie -Ville -> Region
e	Quel est le chiffre d'affaires global mensuel, trimestriel, annuel ?	KPI-Q-e-05 : Chiffre d'Affaires Global Net	Suivre la performance financière globale de l'entreprise.	Financier	CA Net	$SUM(CA \text{ Net})$	Temps	Mois -> Trimestre -> Année

f	Quelles sont les promotions qui rapportent le plus à l'entreprise ?	KPI-Q-f-06 : CA Net Attribué à la Promotion	Évaluer l'impact financier direct des campagnes promotionnelles.	Financier	CA Net	SUM(CA Net) WHERE NomPromo IS NOT NULL	Promotion	Nom Promo
g	Comment les ventes se comportent dans les différents canaux de vente ?	KPI-Q-g-07 : Part du CA par Canal	Déterminer la contribution relative de chaque mode de commande au CA total.	Taux	CA Net	(CA Net par ModeCommande / CA Net Global) * 100	Lieu	ModeCommande
h	Les promotions sont-elles efficaces ?	KPI-Q-h-08 : Taux d'Attachement ou d'utilisation de la Promotion	Mesurer la fréquence à laquelle la promotion est utilisée (adoption).	Taux	Nombre de Transactions	(Transactions avec Promo / Transactions Total) * 100	Promotion	Nom Promo
i	Quel est le panier moyen selon le canal de vente ?	KPI-Q-i-09 : Panier Moyen Net	Évaluer la valeur moyenne de chaque transaction selon le canal.	Financier	- CA Net - Nombre de Transactions	SUM(CA Net) / SUM(Nombre de Transactions) ou SUM(CA Net) / COUNT(ID_Vente)	Lieu (Canal)	ModeCommande
j	Quelle est la marge bénéficiaire nette par catégorie de produits et par région ?	KPI-Q-j-10 : Marge Bénéficiaire Nette	Calculer le profit réel après coûts et remises.	Financier	- PrixUnitaire - Quantite, - TauxPromo, - CoutAchat	SUM((PrixUnitaire * (1 - TauxPromo/100) - CoutAchat) * Quantite)	Produit Lieu	-Produit -> Catégorie -Ville -> Région
k	Quel est le comportement d'achat par type de client (Particulier, Revendeur, Grossiste)	KPI-Q-k-11 : CA par Type de Client	Analyser la valeur par segment (Revendeur, Particulier).	Financier	CA Net	SUM(CA Net)	Client	TypeClient

	?							
l	Quelle est la performance comparée des canaux de vente (ModeCommande) en termes de volume ?	KPI-Q-l-12: Volume par Mode de Commande	Mesurer la performance logistique des canaux.	Volume	-Quantite	SUM(Quantite)	Lieu (Canal)	ModeCommande
m	Quel est l'impact réel des taux de promotion sur la marge brute ?	KPI-Q-m-13: ROI de la Promotion	Vérifier si le volume compense la perte de valeur unitaire.	Financier	-Marge - Quantite	(Marge avec Promo / Marge moyenne sans Promo)	Promotion	TauxPromo
n	Quelle est la fidélité géographique des clients (répétition d'achat par Ville/Région) ?	KPI-Q-n-14: Taux de Rétention Client	Mesurer la récurrence des achats par client.	Taux	Nb Transactions	COUNT(ID_Vente) / COUNT(DISTINCT NomClient)	- Client - Lieu	-TypeClient -Ville -> Region

3. Regroupement de KPI par Thème

Les KPIs sont regroupés en trois thèmes principaux pour organiser le tableau de bord décisionnel.

Thème	Objectif	KPIs Correspondants
Performance Financière Globale et Rentabilité	Suivi du revenu global, de la rentabilité, des marges bénéficiaires, de la croissance et de la tendance temporelle.	-KPI-Q-a-01 -KPI-Q-c-03 -KPI-Q-e-05 - KPI-Q-j-10 - KPI-Q-m-13
Analyse de la Demande et du Volume(Stocks)	Compréhension fine des volumes vendus par produit, catégorie et localisation géographique	-KPI-Q-b-02 -KPI-Q-d-04 -KPI-Q-n-14

Efficacité Commerciale et des Canaux	Évaluation de la performance des interfaces de vente (Modes de commande) et du comportement d'achat (Panier moyen).	-KPI-Q-g-07 -KPI-Q-i-09 -KPI-Q-k-11 -KPI-Q-l-12
Impact des Campagnes Promotionnelles	Mesure de l'attractivité et de la rentabilité spécifique des offres promotionnelles nommées.	- KPI-Q-f-06 - KPI-Q-h-08

4. Liste des Faits, Mesures et Dimensions

Le processus métier modélisé est la **Vente** .

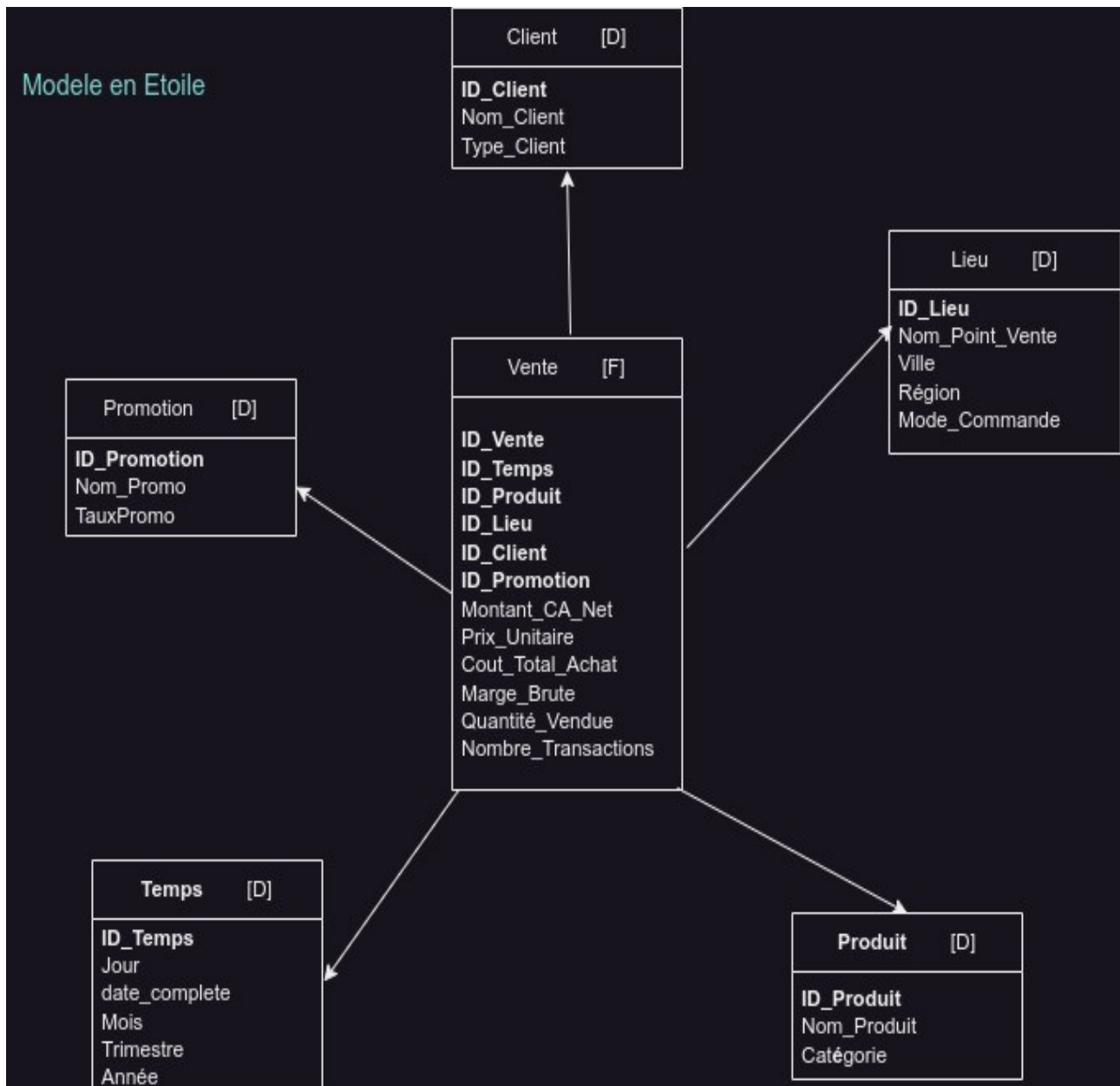
Type	Nom de l'Élément	Origine/Rôle
Fait Principal	Vente	Représente chaque ligne de transaction issue de la source de données.
Mesure	Montant_CA_Net	Additif. (Calculé : Prix Unitaire x Quantité x (1 - TauxPromo/100)).
	Cout_Total_Achat	Additif. (Calculé : CoutAchat * Quantite).
	Marge_Brute	Additif. (Calculé : Montant_CA_Net - Cout_Total_Achat).
	Quantité_Vendue	Additif. Quantité de produit vendu.
	Nombre_Transactions	Additif. Compteur = 1. Utilisé pour le calcul du Panier Moyen.
Dimension	Dim_Temps	Basée sur DateVente. Granularité : Jour (Hiérarchie : Jour → Mois → Trimestre → Année).
	Dim_Produit	Produit, Catégorie.
	Dim_Lieu	PointVente, Ville, Région. Permet l'analyse par localisation mais aussi de la manierer donc ces lieux confirment la commande (Basée sur ModeCommande (Site Web, App, Magasin, Téléphone))
	Dim_Client	NomClient, TypeClient
	Dim_Promotion	NomPromo, TauxPromo

5. Schéma de l'Entrepôt de Données (Question 4)

Le schéma est basé sur le modèle **en Étoile** (simplicité et performance) qui est ensuite décliné en **Flocon de Neige** (normalisation).

5.1. Définir un Modèle en Étoile

Le Modèle en Étoile est le modèle de référence. Il se compose d'une seule table de faits **Vente** au centre, reliée directement à toutes les dimensions.



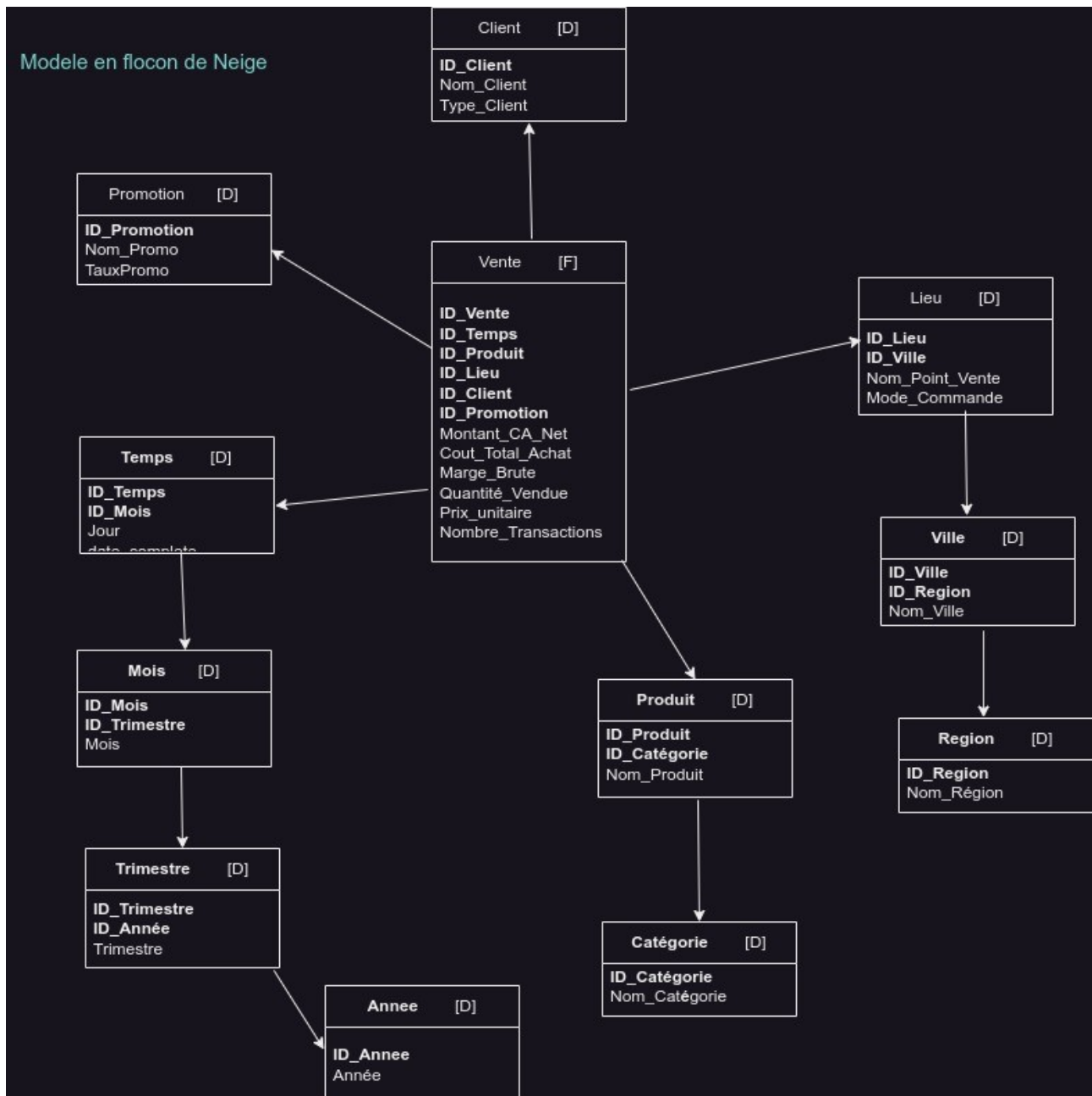
5.2. Identifier les Hiérarchies des Dimensions

Les hiérarchies permettent de naviguer dans les données (drill-down / roll-up) :

- **Dim_Temps** : Jour → Mois → Trimestre → Année
- **Dim_Produit** : Produit → Catégorie
- **Dim_Lieu** : Nom du Point de Vente → Ville → Région

5.3. Construire le Modèle en Flocon de Neige

Le Modèle en Flocon de Neige normalise le Modèle en Étoile en décomposant les dimensions qui possèdent des attributs hiérarchiques.



La conception hiérarchique du Flocon de Neige garantit une **faible redondance** des données descriptives tout en restant suffisamment flexible pour répondre à l'ensemble des besoins analytiques exprimés par « Les Débrouillards ».

NB : Dans ce schéma , [F] est mis pour **Fait** représentant ainsi la table de fait et [D] est mis pour **Dimension** , représentant ainsi les axes d'analyses.

B. Implémentation et Alimentation du datawarehouse (version Flocon)

1. Création de la base de données opérationnelle (vente_DB)

L'objectif était de mettre en place une zone de stockage temporaire pour les données brutes.

- **Action** : Nous avons créé la base vente_DB et la table vente_op.

- **Méthode :** Le schéma de cette table est "à plat", calqué sur la structure du fichier CSV. Pour l'alimentation, nous avons utilisé un script Python (Pandas) afin d'automatiser l'importation des 10 000 lignes. Cela permet de garantir que les types de données (dates, nombres décimaux) sont correctement interprétés par MySQL dès le départ.

2. Création et réajustement du DW (vente_DW)

Pour cette étape, nous avons transformé la structure plate en un **modèle en flocon** rigoureux. Voici comment nous avons procédé :

2.1. Architecture des dimensions (La normalisation)

Contrairement à un schéma en étoile simple, nous avons éclaté les dimensions pour supprimer les redondances :

- **Dimension Lieu :** Nous avons créé trois tables liées (Lieu → Ville → Region). Cela permet une analyse granulaire : on peut observer les ventes par point de vente précis, par ville ou par région entière.
- **Dimension Produit :** Séparée en deux tables (Produit → Catégorie) pour hiérarchiser le catalogue.
- **Dimension Temps :** C'est le point le plus technique. Nous avons créé une hiérarchie complète (Temps → Mois → Trimestre → Année).

2.2. Gestion des Identifiants (IDs) et Clés Intelligentes

Une attention particulière a été portée à la génération des clés primaires :

- **Auto-incrémentation :** Pour les tables comme Client, Lieu ou Produit etc sauf les tables de la hiérarchie temps, nous avons utilisé l'auto-incrément. Nous avons noté que si des erreurs surviennent lors des tests, quand nous relançons, les IDs peuvent ne pas commencer à 1. Nous avons résolu cela avec la commande `DELETE ... ALTER TABLE ... AUTO_INCREMENT = 1` pour garantir un DW propre.
- **Clés Intelligentes (Smart Keys) :** Pour la dimension Temps, nous n'avons pas utilisé l'auto-incrément. Les IDs sont calculés (ex: 201901 pour Janvier 2019). Cela permet de garder une logique chronologique et facilite les jointures temporelles.

2.3. Conception de la Table de Faits (Vente)

La table de faits a été conçue pour centraliser les mesures de performance.

- **Les Clés Étrangères :** Elle contient les IDs pointant vers le niveau le plus fin de chaque dimension (ex: `id_produit` et non `id_categorie`).
- **Les Mesures Calculées :** Nous avons fait le choix stratégique de ne pas stocker que les données brutes. Nous avons calculé directement :
 - **Le Coût d'Achat Total :** (Quantité × Coût D'achat), indispensable pour les agrégations globales.

- **Le Montant CA Net** : Prise en compte immédiate des promotions.
- **La Marge Brute** : Différence directe entre le CA Net et le Coût d'achat Total, permettant d'évaluer la rentabilité par produit ou par ville instantanément.

Pour garantir la robustesse du Data Warehouse, l'implémentation ne s'est pas limitée à la création des tables ; nous avons mis en place une **intégrité référentielle stricte** :

- **Clés Primaires (PK)** : Chaque table de dimension possède une clé unique. Pour les dimensions normalisées (Flocon), cela garantit qu'une ville appartient à une et une seule région par exemple.
- **Clés Étrangères (FK)** : La table de faits `Vente` est liée aux dimensions par des contraintes FOREIGN KEY. Cela empêche l'insertion d'une vente pour un produit ou un client qui n'existerait pas dans les tables de dimensions.
- **Intégrité de Domaine** : Nous avons utilisé des types de données précis (ex: DECIMAL (15, 2) pour les montants financiers) afin d'éviter les erreurs d'arrondi sur les 10 000 lignes de transactions.

3. ETL 1 : Du CSV à `vente_db` en Python

Cette étape constitue la première phase de notre chaîne décisionnelle. Elle consiste à extraire les données du fichier source pour les charger dans notre base opérationnelle.

3.1. Mise en œuvre du processus ETL

Le choix de **Python** pour cette étape est stratégique : il permet de manipuler les données avant même qu'elles ne touchent la base de données.

Méthodologie appliquée :

1. **Extraction (Extract)** : Lecture du fichier CSV à l'aide de la bibliothèque `Pandas`. Cette étape permet de charger les 10 000 lignes en mémoire vive (`DataFrame`) pour un traitement ultra-rapide.
2. **Transformation légère (Transform)** :
 - **Nettoyage** : Vérification de la présence de valeurs nulles ou de lignes vides.
 - **Typage** : Conversion de la colonne `DateVente` au format `datetime` pour s'assurer que SQL la reconnaisse comme une date valide.
 - **Normalisation textuelle** : Suppression des espaces inutiles dans les noms de produits ou de villes pour éviter les doublons lors des futures jointures.
3. **Chargement (Load)** : Utilisation de `SQLAlchemy` pour établir la connexion avec MySQL. Le chargement s'effectue via la méthode `to_sql`.

Note technique : Nous avons utilisé l'option `if_exists='replace'` pour la première exécution afin de garantir que la structure de la table `vente_op` est parfaitement synchronisée avec les données du CSV.

Justification de l'approche : Plutôt que d'utiliser l'importateur natif de phpMyAdmin (souvent limité en taille de fichier ou en gestion des types), l'ETL en Python offre un contrôle total.

En cas de données corrompues dans le CSV, le script Python permet d'identifier la ligne exacte qui pose problème avant l'insertion en base de données.

3.2. Optimisation du chargement massif : La stratégie du `chunksize=1000`

Lors de l'alimentation de `vente_op` avec 10 000 lignes, envoyer toutes les données en une seule fois peut saturer la mémoire tampon (buffer) du serveur MySQL ou provoquer une déconnexion.

- **Le mécanisme :** Le paramètre `chunksize=1000` fragmente l'envoi. Python découpe le DataFrame en 10 paquets de 1000 lignes.
- **L'avantage :** Cela réduit la consommation de RAM sur le serveur et permet de valider les transactions au fur et à mesure. Si une erreur survient à la ligne 5001, on sait que les 5000 premières sont déjà sécurisées. C'est une technique indispensable pour le **Big Data**.

L'implémentation de cet ETL 1 ne s'est pas limitée à un simple transfert de données. Elle a intégré des problématiques de **gestion de flux** (via le chunking), de **nettoyage d'index** (via la réinitialisation des auto-incréments) et de **pré-calcul décisionnel**.

4. ETL 2 : De `vente_db` à `vente_dw` en SQL

Cette phase constitue le cœur du processus de transformation. L'objectif est de passer d'une structure plate et dénormalisée (`vente_op`) vers une structure dimensionnelle en flocon (`vente_dw`).

4.1. Mise en œuvre du processus ETL en SQL

Contrairement à l'ETL 1 qui était un simple transfert, l'ETL 2 repose sur une logique de **mappage et de transformation**. Nous avons décomposé ce processus en deux grandes étapes pour garantir l'intégrité des données.

4.1.1. Alimentation des Dimensions (Logique ascendante)

Pour respecter les contraintes de clés étrangères (Foreign Keys), nous avons alimenté les tables en suivant la hiérarchie du flocon, des tables "parentes" vers les tables "enfants" :

- **Niveau 1 (Indépendant) :** Alimentation des tables `Region`, `Categorie`, `Client`, `Promotion` et `Annee` en utilisant la clause `SELECT DISTINCT`. Cela permet d'isoler les membres uniques de chaque dimension.
- **Niveau 2 (Dépendant) :** Alimentation des tables `Ville`, `Produit` et `Trimestre`. Ici, le SQL doit effectuer une jointure vers les tables du Niveau 1 pour récupérer l'ID correspondant (ex: associer une Ville à l'ID de sa Région).
- **Niveau 3 :** Alimentation des tables `Lieu` et `Mois`.

4.1.2. Alimentation de la Table de Faits (Vente)

C'est l'étape la plus complexe car elle nécessite de transformer les noms textuels de la base opérationnelle en IDs numériques tout en calculant les mesures de performance.

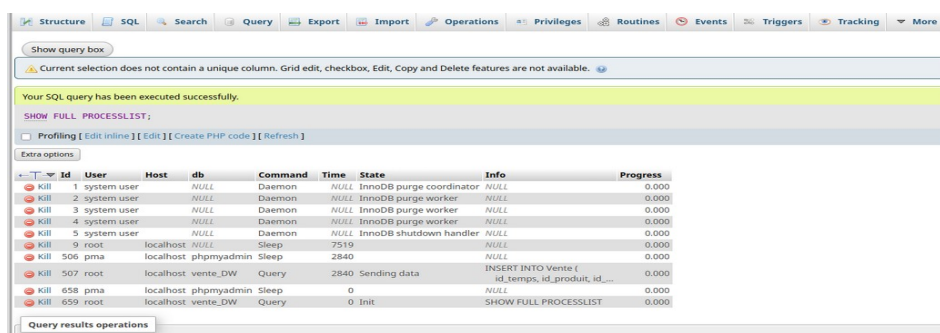
Méthodologie de transformation :

- **Mappage des IDs** : Nous avons utilisé des jointures (INNER JOIN) entre `vente_op` et toutes les tables de dimensions. Cela permet de remplacer, par exemple, le nom du client par son `id_client` unique.
- **Calcul des KPIs en temps réel** : Les mesures n'ont pas été simplement copiées, elles ont été calculées au sein de la requête INSERT :
 - **Coût d'achat total** : $[Quantite \times CoutAchat]$
 - **Montant CA Net** : $[(PrixUnitaire \times Quantite) \times (1 - TauxPromo)]$
 - **Marge Brute** : $[CANet - CoutAchatTotal]$

4.2. Optimisation et résolution des problèmes techniques

Lors de l'alimentation des 10 000 lignes, nous avons rencontré des erreurs de type `#1205 - Lock wait timeout exceeded`. Pour résoudre cela et assurer un processus robuste, nous avons mis en œuvre les solutions suivantes :

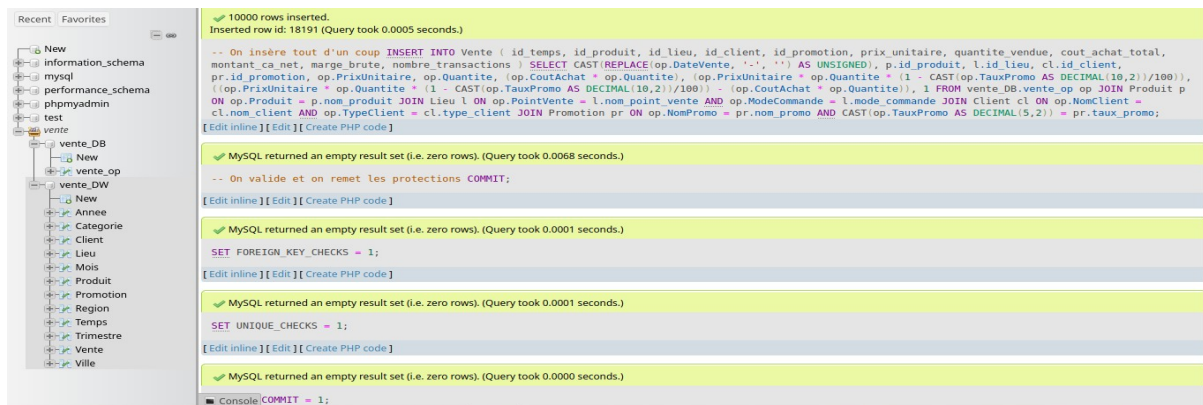
1. **Gestion des transactions** : Utilisation de `SET AUTOCOMMIT = 0` et `COMMIT`. Au lieu de forcer le serveur à écrire 10 000 fois sur le disque, nous avons groupé l'opération en une seule transaction massive, ce qui a drastiquement réduit le temps de traitement.
2. **Désactivation des contraintes** : Utilisation de `SET FOREIGN_KEY_CHECKS = 0` durant l'insertion massive pour éviter que le serveur ne sature en vérifiant les index à chaque ligne.
3. **Nettoyage des processus** : Utilisation de la commande `SHOW PROCESSLIST` et `KILL` pour libérer les ressources du serveur occupées par des requêtes en attente, garantissant ainsi un environnement propre pour l'ETL.



Id	User	Host	db	Command	Time	State	Info	Progress
Kill 1	system user	NULL	NULL	Daemon	NULL	InnoDB purge coordinator	NULL	0.000
Kill 2	system user	NULL	NULL	Daemon	NULL	InnoDB purge worker	NULL	0.000
Kill 3	system user	NULL	NULL	Daemon	NULL	InnoDB purge worker	NULL	0.000
Kill 4	system user	NULL	NULL	Daemon	NULL	InnoDB purge worker	NULL	0.000
Kill 5	system user	NULL	NULL	Daemon	NULL	InnoDB shutdown handler	NULL	0.000
Kill 9	root	localhost	NULL	Sleep	7519	NULL	NULL	0.000
Kill 506	pma	localhost	phpmyadmin	Sleep	2840	NULL	NULL	0.000
Kill 507	root	localhost	vente_DW	Query	2840	Sending data	INSERT INTO Vente (	0.000
Kill 658	pma	localhost	phpmyadmin	Sleep	0	NULL	id_temps, id_produit, id_...	0.000
Kill 659	root	localhost	vente_DW	Query	0	Init	SHOW FULL PROCESSLIST	0.000

L'ETL 2 en SQL a permis de démontrer que la transformation de données peut être gérée directement par le moteur de base de données. Cependant, cette méthode a révélé une forte dépendance aux ressources du serveur et à la gestion des verrous (locks), ce qui justifie l'exploration d'une alternative avec Python.

✓ Résultat



5. ETL 3 : De vente_db à vente_dw en PYTHON

5.1. Méthodologie et Choix Techniques

Pour cette approche, nous avons substitué les requêtes SQL procédurales par un script Python orienté objet utilisant deux piliers technologiques :

- **SQLAlchemy** : Pour l'abstraction de la base de données. Il permet d'ouvrir des sessions de connexion persistantes et optimisées, évitant la surcharge de reconnexion pour chaque table.
- **Pandas** : Pour le traitement **In-Memory**. Les données sont extraites une seule fois de la source, puis transformées en structures appelées **DataFrames** pour une manipulation ultra-rapide.

5.2. Orchestration du Schéma en Flocon

Le script gère l'alimentation de manière atomique et ordonnée pour respecter les contraintes d'intégrité :

1. **Dédoublonnage Vectorisé** : À l'aide de la méthode `.drop_duplicates()`, Python scanne les 10 000 lignes instantanément pour extraire les entités uniques des dimensions (Régions, Catégories, Clients, etc).
2. **Mappage d'Index (Lookup)** : Au lieu d'utiliser des JOIN SQL qui sollicitent le disque dur, Python crée des tables de correspondance en mémoire vive. Faire correspondre un "Nom de Produit" à son "ID" se fait alors en un temps quasi-nul.
3. **Intégrité des données** : Un typage strict a été appliqué (équivalent au CAST SQL) sur les taux de promotion pour assurer une jointure parfaite entre les données opérationnelles et la dimension `Promotion`.
4. **Expansion Temporelle** : Nous avons exploité la puissance de l'objet `datetime` de Pandas. À partir d'une simple colonne `DateVente`, le script dérive automatiquement les attributs de la hiérarchie (Année, Trimestre, Mois, Jour), garantissant une granularité temporelle parfaite sans calcul complexe côté serveur.

5.3. Intégrité et Calcul des KPIs

Le calcul des mesures financières a été optimisé par des opérations vectorielles :

- **Précision Numérique** : L'utilisation de `pd.to_numeric(errors='coerce')` sur le `TauxPromo` agit comme un filtre de qualité (Data Cleaning). Il convertit les types mixtes et assure que les calculs du **CA Net** et de la **Marge Brute** ne produiront pas d'erreurs d'arrondi ou de type, ce qui est souvent le point faible des scripts SQL natifs.
- **Formules métier** : Les indicateurs comme la `marge_brute`, `montant_ca_net`, `cout_achat_total` sont calculés sur l'ensemble du DataFrame d'un seul coup, ce qui est mathématiquement plus efficient que le traitement ligne à ligne de MySQL.

Le script s'est exécuté en **moins de 5 secondes**, marquant une rupture nette avec l'heure d'attente de l'ETL SQL.

✓ Résultat

```

/opt/lampp/bin/mysql ~ X ~/KAMDOM_NGUETCHESSI_MERVEILLE_P... X
(venv) cognisca@blackqueen ~/KAMDOM_NGUETCHESSI_MERVEILLE_PRISCA> python3 etl.py
Démarrage de l'ETL Python (Respect strict du schéma Flocon)??
Calcul de la table de faits et des KPIs...
Terminé ! Toutes les tables du flocon sont remplies avec succès.
(venv) cognisca@blackqueen ~/KAMDOM_NGUETCHESSI_MERVEILLE_PRISCA>

```

Recent Favorites

New

information_schema

mysql

performance_schema

phoenixadmin

test

vente

vente_DB

vente_op

vente_DW

New

Annee

Category

Client

Lieu

Mois

Produit

Promotion

Region

Temps

Trimestre

Vente

Ville

Showing rows 0 - 24 (10000 total, Query took 0.0005 seconds)

SELECT * FROM 'Vente'

Profiling

Edit raw

Edit

Explain SQL

Change PHP code

Refresh

1

>

>>

Number of rows: 25

Filter rows:

Search this table

Sort by key: None

Extra options

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

id_vente_pk

id_temps

id_produit

id_lieu

id_client

id_promotion

prix_unitaire

quantite_vendue

cout_achat_total

montant_ca_net

marge_brute

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

1

	id_vente_pk	id_temps	id_produit	id_lieu	id_client	id_promotion	prix_unitaire	quantite_vendue	cout_achat_total	montant_ca_net	marge_brute	nombre_transactions
1	20190213	1	1	1	1	1	4629.00	65	273000.00	300885.00	27885.00	1
2	20181103	2	2	2	2	1	1515.00	93	111600.00	140895.00	29295.00	1
3	20250725	3	3	3	1	1	1946.00	80	104000.00	155680.00	51680.00	1
4	20181008	4	4	4	3	2	1874.00	81	101250.00	113845.50	12595.50	1
5	20221223	5	5	5	4	3	2532.00	96	192000.00	206611.20	14611.20	1
6	20220108	5	6	5	1	1	2329.00	6	12000.00	13974.00	1974.00	1
7	20191128	6	7	6	6	1	1949.00	32	48000.00	62368.00	14368.00	1
8	20200809	7	8	7	1	1	5424.00	49	220500.00	265776.00	45276.00	1
9	20250807	3	9	8	1	1	2070.00	33	42900.00	68310.00	25410.00	1
10	20250928	8	10	9	1	1	2175.00	57	102600.00	123975.00	21375.00	1
11	20220412	2	3	10	4	1809.00	26	31200.00	37627.20	6427.20		1
12	20210129	9	11	11	1	3088.00	38	83600.00	117344.00	33744.00		1
13	20241022	1	12	12	1	5120.00	96	403200.00	491520.00	88320.00		1
14	20210619	10	13	13	3	2251.00	25	32500.00	47833.75	15333.75		1
15	20210831	1	2	14	1	5093.00	47	197400.00	239371.00	41971.00		1
16	20191105	8	14	15	1	2305.00	17	30600.00	39185.00	8585.00		1
17	20181105	11	15	16	1	2402.00	96	172800.00	230592.00	57792.00		1
18	20190105	10	16	17	1	2292.00	93	120900.00	213156.00	92256.00		1
19	20250718	12	17	18	1	3070.00	69	172500.00	211830.00	39330.00		1
20	20180413	4	18	19	1	2203.00	48	60000.00	105744.00	45744.00		1
21	20250413	13	19	20	1	3024.00	29	60900.00	87696.00	26796.00		1

6. Analyse comparative et Discussion (ETL 2 vs ETL 3)

Cette étude comparative met en lumière deux philosophies radicalement différentes pour l'intégration de données dans un environnement décisionnel. L'expérimentation menée sur un échantillon de 10 000 transactions a permis de quantifier l'efficacité de chaque approche.

6.1 Tableau de synthèse comparative

Critère de comparaison	ETL2 - Approche SQL (Procédurale)	ETL3 - Approche Python (Objet/In-Memory)
Temps d'exécution	Lente (~759 Secondes)	Ultra-rapide (< 5 secondes)
Gestion des ressources	Surcharge le processeur et le disque du serveur SQL.	Utilise la mémoire vive (RAM) de la machine locale.
Complexité du code	Élevée (jointures multiples et imbriquées) à l'instar de la Requête <code>INSERT INTO . . . SELECT</code> avec jointures imbriquées.	Modérée (logique séquentielle et modulaire , code lisible et facile à déboguer).
Flexibilité des données	Rigide (le moindre écart de type bloque la requête).	Rigide (le moindre écart de type bloque la requête).
Scalabilité	Difficile sur de très gros volumes (risques de verrous).	Excellente grâce au chargement par paquets (<i>chunksize</i>).
Couplage	Fortement lié au moteur MySQL.	Flexible et portable (indépendant du SGBD).

6.2. Discussion ETL2 vs ETL3

La différence massive de temps d'exécution s'explique par la gestion des **entrées/sorties (I/O)**.

- En **SQL**, le moteur doit vérifier les contraintes d'intégrité et mettre à jour les index du schéma en flocon à chaque insertion, tout en gérant les verrous sur le disque dur. Pour 10 000 lignes, cette surcharge transactionnelle devient un goulot d'étranglement.
- En **Python**, l'utilisation de la bibliothèque **Pandas** permet d'effectuer les transformations dans l'espace d'adressage de la mémoire vive. Le calcul vectoriel remplace les boucles itératives, ce qui rend le traitement des jointures entre les tables `vente_op` et les dimensions quasi-instantané avant l'envoi groupé vers `vente_DW`.
- **Qualité et Fiabilité** : Python permet d'intégrer une étape de **Data Cleaning** (nettoyage) plus poussée. Par exemple, la gestion des taux de promotion via `to_numeric` permet de transformer des données "sales" en entrées propres pour le Data Warehouse, là où le SQL aurait simplement échoué avec une erreur de syntaxe.

6.3. Suggestions d'évolutions et Extensions du TP : Perspectives d'analyses futures

1. **Reporting Décisionnel Interactif (Power BI/Tableau)** : L'exploitation du schéma en flocon via un outil de Dataviz permettrait de créer des tableaux de bord dynamiques pour piloter la performance commerciale (top produits, saisonnalité des ventes, analyse de la rentabilité par région).

2. **Analyse Prédictive (Machine Learning)** : Sur la base de l'historique stocké dans le Data Warehouse, il serait possible de développer des modèles de prévision de la demande ou de segmentation client (RFM) pour optimiser la gestion des stocks et les campagnes marketing.

Guide de Déploiement Technique

Ce guide détaille les étapes nécessaires pour reproduire l'environnement et exécuter les processus d'alimentation (ETL) du projet.

A. Alimentation du Schéma en Flocon (SQL)

Après avoir créé les bases de données `vente_op` et `vente_DW` via les scripts de structure fournis, l'alimentation de l'entrepôt doit se faire **séquentiellement**. Pour respecter l'intégrité référentielle (clés étrangères), ne lancez pas le script en un seul bloc, mais suivez cet ordre hiérarchique :

1. Niveau 1 : Dimensions indépendantes (Racines)

- Exécuter les blocs : `Region`, `Categorie`, `Client`, `Promotion` et `Annee`.

2. Niveau 2 : Dimensions dépendantes (Niveau 1)

- Exécuter les blocs : `Ville` (dépend de `Région`), `Produit` (dépend de `Catégorie`) et `Trimestre` (dépend de `Année`) et `Mois` (dépend de `Trimestre`)..

3. Niveau 3 : Dimensions dépendantes (Niveau 2)

- Exécuter les blocs : `Lieu` (dépend de `Ville`) et `Temps`

4. Niveau 4 : Faits

- Terminer par la table de faits : `Vente` (nécessite que toutes les autres tables soient remplies).

B. Exécution de l'ETL Python

Le script a été développé dans un environnement virtuel isolé afin de garantir la portabilité et la sécurité du système (conformément à la norme **PEP 668** sur les systèmes Linux récents).

- **Si vous utilisez Anaconda ou un environnement Python déjà configuré** : Vous pouvez lancer directement `python etl.py` après avoir vérifié que `pandas` et `sqlalchemy` sont installés.
 - **Si vous rencontrez une erreur "Externally Managed Environment"** : Veuillez suivre la procédure `venv` détaillée ci-dessous pour isoler l'exécution.
1. **Préparation** : Assurons-nous que tous les éléments (fichier `.csv`, scripts `.py`) se trouvent dans le même répertoire.

2. **Configuration de l'environnement virtuel** : Dans le terminal, tapons ces commandes l'une après l'autre

1. **Créer l'environnement** (bulle isolée) : **python3 -m venv venv**

2. **Activer cette bulle** :

1. **source venv/bin/activate.fish** **# Pour Fish Shell**

2. **source venv/bin/activate** **#Pour Bash/Zsh**

3. **Installation des outils (si nécessaire)** : Si les bibliothèques ne sont pas encore présentes dans notre environnement, installons-les via ces commandes

1. **pip install mysql-connector-python --default-timeout=100**

2. **pip install sqlalchemy --default-timeout=100**

3. **pip install pandas --default-timeout=600**

4. **pip install mysql-connector-python sqlalchemy pandas**

4. **Exécution des scripts** : Une fois l'installation terminée et l'environnement activé (venv), assurez-vous d'être dans le bon répertoire et lancez les commandes suivantes :

1. **Étape 1 : Chargement des données sources** (du fichier .CSV vers la table `vente_op`) : **python3 BI.py**

2. **Étape 2 : Exécution de l'ETL** (Transformation et chargement vers `vente_DW`) : **python3 etl.py**

Note importante : Avant d'exécuter le script Python `etl.py`, assurons-nous de vider les tables en exécutant les commandes `DELETE` et `ALTER TABLE` du script `bi.sql` pour garantir l'intégrité des données.

Conclusion

Ce projet a permis de maîtriser le cycle complet de l'information, de la base opérationnelle à l'entrepôt de données en flocon. L'expérimentation a démontré que si le SQL assure la structure et la persistance des données, Python est le moteur de performance indispensable pour la phase de transformation, réduisant le temps de traitement de façon drastique. En combinant ces deux technologies, nous avons mis en place une architecture robuste capable de transformer 10 000 lignes brutes en indicateurs fiables (CA Net, Marge) prêts à être exploités par les décideurs pour orienter la stratégie de l'entreprise.